
Compiler Design

No	Problem Name
01	Write a C program for developing a lexical analyzer (LA) that will eliminate white spaces from a source program in C and collect numbers.
02	Write a C program for developing a lexical analyzer (LA) that will eliminate white spaces from a source program in C and collect numbers as token and then also display the token value as attribute.
03	Write a C program for developing a lexical analyzer (LA) that will recognize the variables in a source program. ✓
04	Write a C program for developing a lexical analyzer (LA) that will recognize the keywords in a source program.
05	Design a compiler front-end based on syntax-directed translation technique that will function as an infix translator for a language consists of sequence of expressions terminated by semicolon. ✓
✓ 06	Write a program to remove left factoring.
✓ 07	Write a program to check whether a string belongs to the grammar or not. ✓
08	Write a C program to implement Shift-Reduce Parser.
09	Write a C program to implement a recursive descent parser.

1. Write a C program for developing a lexical analyzer (LA) that will eliminate white spaces from a source program in C and collect numbers.

```
#include<iostream>

#include<stdio.h>

using namespace std;

int sign_check(char pre_value){

    if(pre_value=='+'||pre_value=='-'||pre_value=='*'||pre_value=='/'||pre_value=='%'||pre_value=='=')

        return 1;

    else

        return 0;

}

int main()

{

    int n,len,s,i;

    char pre_value=' ';

    string str;

    FILE *f2;

    freopen("1 Input.txt","r",stdin);

    f2=fopen("1 Output.txt","w");

    printf("Collect no. given below\n");

    while(getline(cin,str)){

        len =str.size();

        for(i=0;i<len;i++){

            if(str[i]!=' ')

                putc(str[i],f2);

        }

        putc('\n',f2);

        for(i=0;i<len;i++){

            if(str[i]>='0'&&str[i]<='9'){

                s=str[i]-'0';
```

```

        for(i=i+1;i<len;i++){
            if(str[i]>='0'&&str[i]<='9')
                s=s*10+(str[i]-'0');
            else
                break;
        }
        i--;
        if(pre_value=='['&&str[i+1]==']')
            cout<<s<<endl;
        else if((sign_check(pre_value)||pre_value==' ')&&(sign_check(str[i+1])||str[i+1]==';'||i==len1))
            cout<<s<<endl;
        }
    else
        pre_value=str[i];
    }
}

return 0;
}

```

Input:

```

#include<stdio.h>
#include<iostream>
using namespace std;
int main()
{
    char t,f;
    int n;
    FILE *f1,*f2;
    f1=fopen("basir.txt","r");
    f2=fopen("rajib.txt","w");
    while(t=getc(f1)!=EOF){

```

```

        if(t!="&&isdigit(t))
            putc(t,f2);
        else{
            n=0;
            while(isdigit(t)){
                putc(t,f2);
                n=n*10+(t-'0');
                t=getc(f1);
            }
            if(n){
                putc(t,f2);
                cout<<n<<endl;
            }
        }
    }
}

return 0;
}

```

Output:

```

#include<stdio.h>
#include<iostream>
usingnamespacestd;
intmain()
{
    chart,f;
    intn;
    FILE*f1,*f2;
    f1=fopen("basir.txt","r");
    f2=fopen("rajib.txt","w");
    while(t=getc(f1)!=EOF){

```

```

        if(t!="&&isdigit(t))
            putc(t,f2);
        else{
            n=0;
            while(isdigit(t)){
                putc(t,f2);
                n=n*10+(t-'0');
                t=getc(f1);
            }
            if(n){
                putc(t,f2);
                cout<<n<<endl;
            }
        }
    }
    return 0;
}

```

```

G:\Porasuna\4th Year\ACMy coding\Final\Compiler\1.exe
Collect no. given below
0
10
0
Process returned 0 (0x0)   execution time : 0.208 s
Press any key to continue.

```

2. Write a C program for developing a lexical analyzer (LA) that will eliminate white spaces from a source program in C and collect numbers as token and then also display the token value as attribute.

```
#include<iostream>

#include<stdio.h>

using namespace std;

int punctuation_check(char pre_value){

    if(pre_value=='+'||pre_value=='-'||pre_value=='*'||pre_value=='/'||pre_value=='%')

        return 1;

    else if(pre_value=='['||pre_value==']'||pre_value=='{'||pre_value=='}'||pre_value=='('||pre_value==')')

        return 1;

    else if(pre_value=='"'||pre_value==';'||pre_value==':'||pre_value==','||pre_value=='.'||pre_value=='\\')

        return 1;

    else

        return 0;

}

int keyword(string str){

    if(str=="int"||str=="char"||str=="float"||str=="double"||str=="string"||str=="typedef"||str=="long"||str=="string")

        return 1;

    else if(str=="while"||str=="do"||str=="for"||str=="if"||str=="else"||str=="switch"

||str=="case"||str=="break"||str=="continue")

        return 1;

    else if(str=="const"||str=="goto"||str=="static"||str=="union"||str=="return"||str=="default"

||str=="short"||str=="signed"||str=="unsigned"||str=="void")

        return 1;

    else

        return 0;

}

int sign_check(char pre_value){

    if(pre_value=='+'||pre_value=='-'||pre_value=='*'||pre_value=='/'||pre_value=='=')
```

```

        return 1;
    else
        return 0;
}

int main()
{
    char t,f;
    int n,checks,s,len;
    string str,stt;
    FILE *f2;
    freopen("2 Input.txt","r",stdin);
    char pre_value='1';
    cout<<"token   token value as attributes\n";
    cout<<"=====\\n";
    while(getline(cin,str)){
        pre_value='1';
        stt="";
        len=str.size();
        for(int i=0;i<len;i++){
            if(punchuation_check(str[i])){
                cout<<"Punchuation "<<str[i]<<"\\n";
                pre_value=str[i];
            }
            else if(str[i]=='<' or str[i]=='>' or str[i]=='='){
                if(i+1<len){
                    if(str[i+1]=='='){
                        cout<<"CO      "<<str[i]<<str[i+1]<<endl;
                        i++;
                    }
                }
                else

```



```

        cout<<"RO        "<<str[i]<<endl;
    }
    pre_value=str[i];
}
else if(str[i]==' ')
    pre_value=str[i];
else if((str[i]>='0' and str[i]<='9') and sign_check(pre_value)){
    s=0;
    s=s*10+str[i]-'0';
    for(i=i+1;i<len;i++){
        if((str[i]>='0' and str[i]<='9'))
            s=s*10+str[i]-'0';
        else
            break;
    }
    i--;
    if(pre_value=='[' and str[i+1]==']')
        cout<<"num        "<<s<<"\n";
    else if( sign_check(pre_value) and i==len-1 )
        cout<<"num        "<<s<<"\n";
    else if( sign_check(pre_value) and (sign_check(str[i+1]) or str[i+1]=='(';))
        cout<<"num        "<<s<<"\n";
}
else{
    stt+=str[i];
    if(punctuation_check(str[i+1]) or str[i+1]==' ' or str[i+1]=='=' or str[i+1]=='<'){
        if(keyword (stt))
            cout<<"Keyword    "<<stt<<"\n";
        else
            cout<<"ID        "<<stt<<"\n";
    }
}

```

```
        stt="";  
    }  
    pre_value=str[i];  
    }  
    }  
    }  
    return 0;  
}
```

Input:

```
char t;  
int n;  
FILE *f1;  
f1=fopen("Input.txt","r");  
while(t=getc(f1)){  
    if(n  
        putc(t,f2);  
}
```

Output:

```
"G:\Porasuna\4th Year\ACMy coding\Final\Compiler\2.exe"
token    token value as attributes
=====
char      char
ID         t
Punctuation ;
int       int
ID         n
Punctuation ;
ID         FILE
Punctuation *
ID         f1
Punctuation ;
ID         f1
RE         =
ID         fopen
Punctuation (
Punctuation "
ID         Input
Punctuation .
ID         txt
Punctuation "
Punctuation ,
Punctuation "
ID         r
Punctuation "
Punctuation )
Punctuation ;
while     while
Punctuation (
ID         t
RE         =
ID        getc
Punctuation (
ID         f1
Punctuation )
Punctuation )
Punctuation {
if        if
Punctuation (
ID         n
Punctuation )
ID         putchar
Punctuation (
ID         t
Punctuation ,
ID         f2
Punctuation )
Punctuation ;
Punctuation }
```

Process returned 0 (0x0) execution time : 0.242 s
Press any key to continue.

3. Write a C program for developing a lexical analyzer (LA) that will recognize the variables in a source program.

```
#include<stdio.h>

#include<iostream>

using namespace std;

int check(string str){

    if(str=="int"||str=="char"||str=="float"||str=="double"||str=="long"||str=="long
long"||str=="signed"||str=="unsigned")

        return 1;

    else

        return 0;

}

int main()

{

    freopen("3 Input.txt","r",stdin);

    string str;

    while(cin>>str){

        if(check(str)){

            cout<<str<<" variable= ";

            int k=0;

            while(1){

                char ch=getchar();

                if(ch==';')

                    break;

                if(k==0&&ch!=' ')

                    cout<<ch;

                if(ch=='=')

                    k=1;

                if(ch==',')
```

```
        k=0;

    }

    cout<<"."<<endl;

}

}

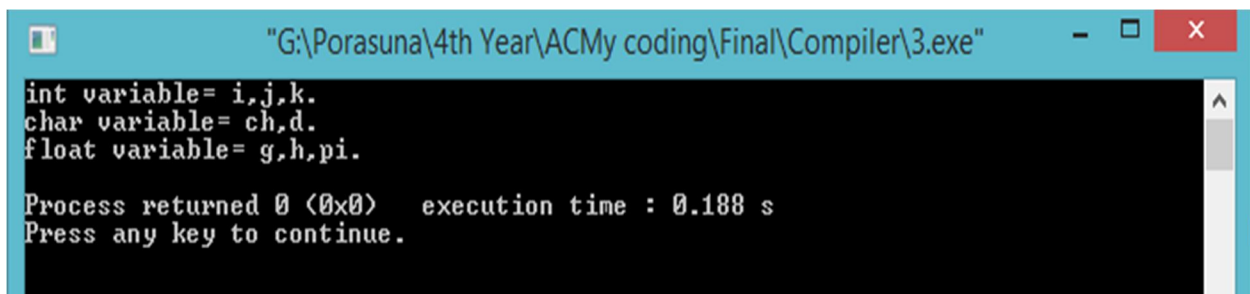
return 0;

}
```

Input:

```
void main()
{
    int i,j,k;
    char ch,d;
    float g,h,pi;
    for(i=0;i<5;i++)
        printf("%d ",i);
    return 0;
}
```

Output:



```
"G:\Porasuna\4th Year\ACMy coding\Final\Compiler\3.exe"
int variable= i,j,k.
char variable= ch,d.
float variable= g,h,pi.
Process returned 0 (0x0)   execution time : 0.188 s
Press any key to continue.
```

4. Write a C program for developing a lexical analyzer (LA) that will recognize the keywords in a source program.

```
#include<bits/stdc++.h>

using namespace std;

int check(string s){

    if(s=="auto"||s=="int"||s=="char"||s=="long"||s=="float"||s=="double"||s=="struct"||s=="if"||s=="else"||s=="break"||s=="continue"||

    s=="while"||s=="do"||s=="for"||s=="return"||s=="signed"||s=="unsigned"||s=="default"||s=="goto"||s=="case"||s=="sizeof"||s=="short"||

        s=="switch"||s=="void"||s=="static"||s=="typedef")

        return 1;

    else

        return 0;

}

int main()

{

    char t,f,ch;

    int n,k;

    int anu;

    string st;

    freopen("9 Input.txt","r",stdin);

    char pre_value='1';

    printf("Keywords are given bellow\n\n");

    while(cin>>st){

        if(check(st)){

            cout<<st<<" is a keyword\n";

        }

    }

}
```

```
    return 0;
}

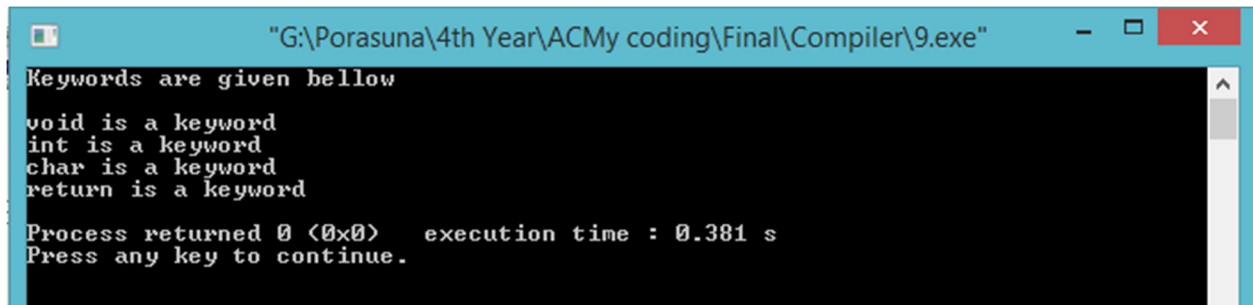
Input:

#include<iostream>

using namespace std;

void main()
{
    int i,a,b,c;
    char ch;
    string str;
    for(i=1;i<=15;i++)
        cout<<"The number is: "<<i<<endl;
    return 0;
}
```

Output:



```
"G:\Porasuna\4th Year\ACMy coding\Final\Compiler\9.exe"
Keywords are given bellow
void is a keyword
int is a keyword
char is a keyword
return is a keyword
Process returned 0 (0x0) execution time : 0.381 s
Press any key to continue.
```

5. Design a compiler front-end based on syntax-directed translation technique that will function as an infix translator for a language consists of sequence of expressions terminated by semicolon.

```
#include<bits/stdc++.h>

using namespace std;

int len,i;

char ch;

void infix(string str){

    stack<char> stack_string;

    str='('+str+')';

    len=str.size();

    for(i=0;i<len;i++){

        if(str[i]>='a'&& str[i]<='z')

            cout<<str[i];

        else if(str[i]=='(')

            stack_string.push('(');

        else if(str[i]==')'){

            while(stack_string.top()!='('){

                cout<<stack_string.top();

                stack_string.pop();

            }

            stack_string.pop();

        }

        else if(str[i]=='+'||str[i]=='-'){

            while(1){

                ch=stack_string.top();

                if(ch=='+'||ch=='-')

                    break;
```



```

        else if(ch=='(')
            break;

        cout<<ch;

        stack_string.pop();
    }

    stack_string.push(str[i]);
}

else if(str[i]=='*'||str[i]=='/'){
    while(1){
        ch=stack_string.top();

        if(ch=='+'||ch=='-'||ch=='*'||ch=='/')
            break;

        else if(ch=='(')
            break;

        cout<<ch;

        stack_string.pop();
    }

    stack_string.push(str[i]);
}

else if(str[i]=='^')
    stack_string.push(str[i]);
}
}

int main()
{
    string str;

    cout<<"Enter the expression: ";

    cin>>str;

    cout<<"\n=====The expression in infix notation=====\\n\\n";

```

```

infix(str);

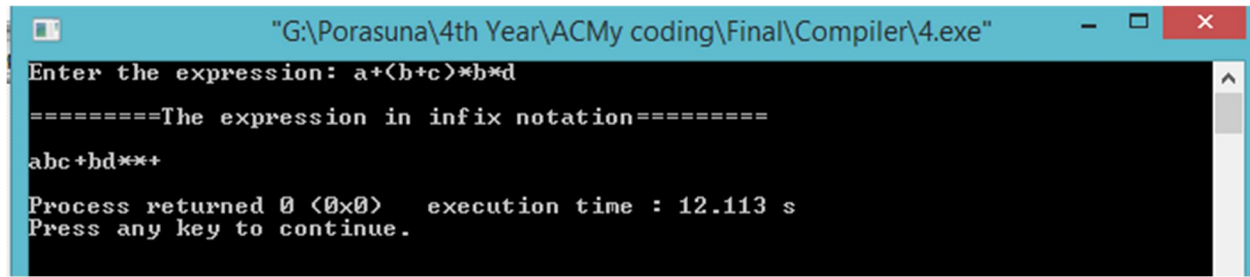
cout<<endl;

return 0;

}

```

Input & Output:



```

G:\Porasuna\4th Year\ACMy coding\Final\Compiler\4.exe
Enter the expression: a+(b+c)*b*d
====The expression in infix notation====
abc+bd**+
Process returned 0 (0x0)   execution time : 12.113 s
Press any key to continue.

```

6. Write a program to remove left factoring.

```

#include<bits/stdc++.h>

using namespace std;

int main()
{
    int len,i,j,k,kk,check=0;
    string str;
    while(1){
        cout<<"Enter the string: ";
        cin>>str;
        len=str.size();

        for(i=0;i<len;i++){
            if(str[i]=='\n')
                break;
        }
    }
}

```

```
j=i;
kk=j+1;

for(k=0;k<len;k++,kk++){
    if(str[k]!=str[kk]){
        check=1;
        break;
    }
}
```

```
if(check && k!=0){
    cout<<"A-> ";
    for(i=0;i<k;i++){
        cout<<str[i];
    }
    cout<<"A'"<<endl;
    cout<<"A'-> ";
}
```

```
if(k==j)
    cout<<"#";
```

```
for(i=k;i<j;i++){
    cout<<str[i];
}
cout<<"|";
```

```
for(i=j+k+1;i<len;i++){
    cout<<str[i];
}
cout<<endl;
}else{
    cout<<"A-> ";
    for(i=0;i<j+1;i++){
```

```
        cout<<str[i];  
    cout<<"A"<<endl;  
    cout<<"A'-> ";  
  
    for(i=j+1;i<len;i++)  
        cout<<str[i];  
    cout<<endl;  
}  
cout<<endl;  
}  
return 0;  
}
```

Input & Output:



The screenshot shows a terminal window with a black background and white text. It displays two separate runs of the program. In the first run, the user enters 'a+b!a+c', and the program outputs 'A-> a+A'' and 'A'-> b!c'. In the second run, the user enters 'iEtS!iEtSeS', and the program outputs 'A-> iEtSA'' and 'A'-> #!eS'. A vertical scrollbar is visible on the right side of the terminal window.

```
Enter the string: a+b!a+c  
A-> a+A'  
A'-> b!c  
  
Enter the string: iEtS!iEtSeS  
A-> iEtSA'  
A'-> #!eS
```

7. Write a program to check whether a string belongs to the grammar or not.

```
#include<bits/stdc++.h>

using namespace std;

int main()
{
    string str;

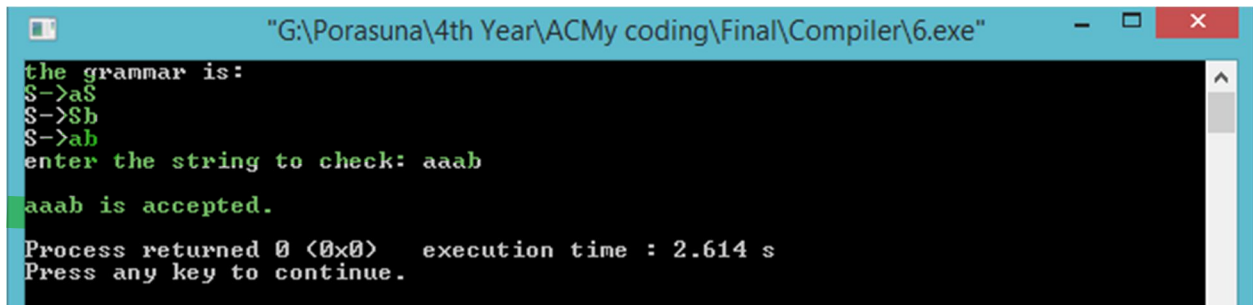
    cout<<"the grammar is: \nS->aS \nS->Sb \nS->ab\n";

    cout<<"enter the string to check: ";

    cin>>str;

    int len=str.size();
    int flag=0,i,a,b;
    for(i=0;i<len;i++){
        if(str[i]!='a'){
            a=i-1;
            break;
        }
    }
    for(i=len-1;i>=0;i--){
        if(str[i]!='b'){
            b=i+1;
            break;
        }
    }
    if(a>=0 && b<len && a+1==b)
        flag=1;
    if(flag==1)
        cout<<endl<<str<<" is accepted."<<endl;
    else
        cout<<endl<<str<<" is not accepted."<<endl;
    return 0;}
```

Input & Output:



```
"G:\Porasuna\4th Year\ACMy coding\Final\Compiler\6.exe"
the grammar is:
S->aS
S->Sb
S->ab
enter the string to check: aaab
aaab is accepted.
Process returned 0 (0x0) execution time : 2.614 s
Press any key to continue.
```

8. Write a C program to implement Shift-Reduce Parser.

```
//(id+id)*id
#include<bits/stdc++.h>
#include<string.h>
using namespace std;

string str,shift_action="Shift ",reduce_action="Reduce to E ";
char stuck[100];
int i,j,len,k;

void checking(){
    for(k=0;k<len;k++){
        if(stuck[k]=='i'&&stuck[k+1]=='d'){
            stuck[k]='E';
            stuck[k+1]='\0';
            cout<<"$"<<stuck<<"\t"<<str<<"$\t"<<reduce_action<<endl;
            j++;
        }
    }
    for(k=0;k<len;k++){
        if(stuck[k]=='E' && stuck[k+1]=='+' && stuck[k+2]=='E'){
            stuck[k]='E';
```

```

        stuck[k+1]='\0';
        stuck[k+2]='\0';
        cout<<"$"<<stuck<<"\t"<<str<<"$\t"<<reduce_action<<endl;
        i=i-2;
    }
}

for(k=0;k<len;k++){
    if(stuck[k]=='E'&&stuck[k+1]=='*'&&stuck[k+2]=='E'){
        stuck[k]='E';
        stuck[k+1]='\0';
        stuck[k+1]='\0';
        cout<<"$"<<stuck<<"\t"<<str<<"$\t"<<reduce_action<<endl;
        i=i-2;
    }
}

for(k=0;k<len;k++){
    if(stuck[k]=='('&&stuck[k+1]=='E'&&stuck[k+2]=='')){
        stuck[k]='E';
        stuck[k+1]='\0';
        stuck[k+1]='\0';
        cout<<"$"<<stuck<<"\t"<<str<<"$\t"<<reduce_action<<endl;
        i=i-2;
    }
}

}

int main()
{
    puts("Grammar is:\n\nE->E+E\nE->E*E\nE->(E)\nE->id\n\n");
    cout<<"Enter the input string: ";
    cin>>str;

```

```

len=str.size();

puts("\nStack\tInput\t\tAction");

//cout<<"$t"<<str<<"$"<<"\t\t"<<shift_action;

for(i=0,j=0;j<len;i++,j++){
    if(str[j]=='i'&&str[j+1]=='d'){
        stuck[i]=str[j];
        stuck[i+1]=str[j+1];
        stuck[i+2]='\0';
        str[j]=' ';
        str[j+1]=' ';
        cout<<"$"<<stuck<<"\t"<<str<<"$"<<"\t"<<shift_action<<"id"<<endl;
        checking();
    }
    else{
        stuck[i]=str[j];
        stuck[i+1]='\0';
        str[j]=' ';
        cout<<"$"<<stuck<<"\t"<<str<<"$"<<"\t"<<shift_action<<"symbol"<<endl;
        checking();
    }
}

if(str[len-1]==' ')
    cout<<"$"<<stuck<<"\t"<<str<<"$t"<<"Accept"<<endl;

return 0;
}

```


Input & Output:

```
"G:\Porasuna\4th Year\ACMy coding\Final\Compiler\7.exe"
Grammar is:
E->E+E
E->E*E
E->(E)
E->id

Enter the input string: <id+id>*id

Stack   Input           Action
$(<      id+id)*id$      Shift symbol
$id      +id)*id$        Shift id
$(E      +id)*id$        Reduce to E
$(E+     id)*id$        Shift symbol
$(E+id   )*id$          Shift id
$(E+E    )*id$          Reduce to E
$(E      )*id$          Reduce to E
$(E)     *id$           Shift symbol
$E       *id$           Reduce to E
$E*      id$           Shift symbol
$E*id    $             Shift id
$E*E     $             Reduce to E
$E       $             Reduce to E
$E       $             Accept

Process returned 0 (0x0)   execution time : 8.011 s
Press any key to continue.
```

9. Write a C program to implement a recursive descent parser.

```
//E->TE'
//E'->+TE'|NULL
//T->FT'
//T'->*F'|NULL
//F->(E)|a|b|c

#include<bits/stdc++.h>
using namespace std;
int i,error;
string str;

void E();
void Eds();
void T();
```

```
void Tds();
```

```
void F();
```

```
int main()
```

```
{
```

```
    cout<<"Enter the input string: ";
```

```
    cin>>str;
```

```
    E();
```

```
    if(i==str.size()&&error==0)
```

```
        cout<<endl<<"String is accepted"<<endl;
```

```
    else
```

```
        cout<<endl<<"String is rejected"<<endl;
```

```
    return 0;
```

```
}
```

```
void E(){
```

```
    T();
```

```
    Eds();
```

```
}
```

```
void Eds(){
```

```
    if(str[i]=='+'){
```

```
        i++;
```

```
        T();
```

```
        Eds();
```

```
    }
```

```
}
```

```
void T(){
```

```
    F();
```

```

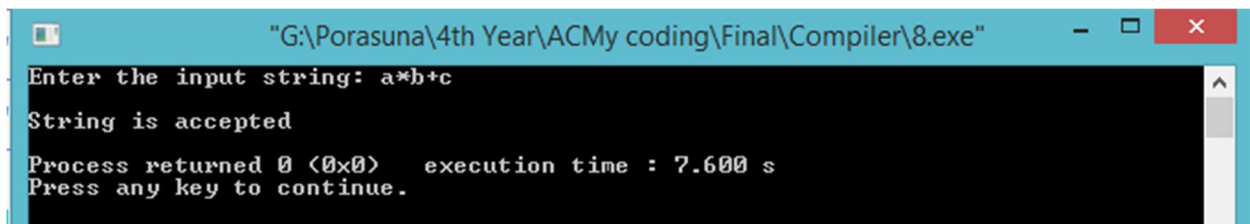
    Tds();
}

void Tds(){
    if(str[i]=='*'){
        i++;
        F();
        Tds();
    }
}

void F(){
    if(str[i]=='a' || str[i]=='b' || str[i]=='c')
        i++;
    else if(str[i]=='('){
        E();
        if(str[i]==')')
            i++;
        else
            error=1;
    }
    else
        error=1;
}

```

Input & Output:



```

"G:\Porasuna\4th Year\ACMy coding\Final\Compiler\8.exe"
Enter the input string: a*b+c
String is accepted
Process returned 0 (0x0)   execution time : 7.600 s
Press any key to continue.

```

System Simulation

No	Problem Name
01	Develop a computer program for determining the value of π and run the simulation for 1000 observations.
02	Write a C Program to simulate pure pursuit problem.
03	Write a C program to generate the area of an irregular figure using monte-carlo method.
04	Write a C program to determine the value of numerical integration using monte-carlo method.
05	Write a C program to determine the value of π by monte-carlo method.
06	Write a C program for Mid-square Method of Generating 4-digit Random Numbers.
07	Write a C program to generate variables from NEGATIVE BINOMIAL distribution having Parameters k (number of successes) and p (probability of success in trial).
08	Write a C program to generate random variable from a Normal distribution having mean mue and standard deviation sigma.
09	

1. Develop a computer program for determining the value of π and run the simulation for 1000 observations.

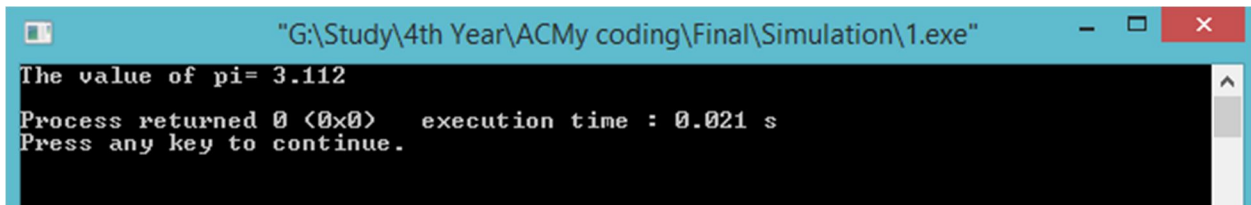
```
#include<bits/stdc++.h>

#include<math.h>

using namespace std;

int main()
{
    float r1,r2,m=0;
    int i,n=1000;
    for(i=1;i<=n;i++){
        r1=rand()/32768.0;
        r2=rand()/32768.0;
        if(r2<=sqrt(1-r1*r1))
            m++;
    }
    cout<<"The value of pi= "<<(m*4)/n<<endl;
    return 0;
}
```

Input & Output:



```
"G:\Study\4th Year\ACMy coding\Final\Simulation\1.exe"
The value of pi= 3.112
Process returned 0 (0x0) execution time : 0.021 s
Press any key to continue.
```

2. Write a C Program to simulate pure pursuit problem.

```
#include<bits/stdc++.h>

using namespace std;

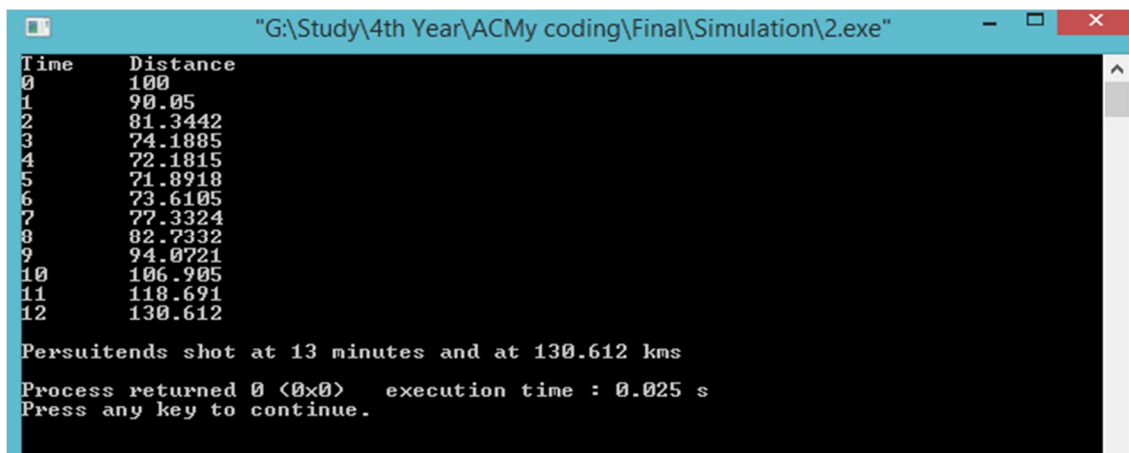
int main()
{
    float xb[]={100,110,120,129,140,149,158,168,179,188,198,209,219,226,234,240};
```

```

float yb[]={0,3,6,10,15,20,26,32,37,34,30,27,23,19,16,14};
float dist,time_limit=12,xf[20],yf[20],s=20;
int i;
xf[0]=0;
yf[0]=0;
cout<<"Time\tDistance\n";
for(i=0;i<=12;i++){
    dist=sqrt(pow(yf[i]-yb[i],2)+pow(xf[i]-xb[i],2));
    cout<<i<<"\t"<<dist<<endl;
    if(i>time_limit)
        printf("\nTarget escapes");
    if(dist<=10)
        break;
    else{
        xf[i+1]=xf[i]+s*(xb[i]-xf[i])/dist;
        yf[i+1]=yf[i]+s*(yb[i]-yf[i])/dist;
    }
}
cout<<"\nPersuitends shot at "<<i<<" minutes and at "<<dist<<" kms\n";
return 0;
}

```

Input & Output:



```

"G:\Study\4th Year\ACMy coding\Final\Simulation\2.exe"
Time    Distance
0       100
1       90.05
2       81.3442
3       74.1885
4       72.1815
5       71.8918
6       73.6105
7       77.3324
8       82.7332
9       94.0721
10      106.905
11      118.691
12      130.612

Persuitends shot at 13 minutes and at 130.612 kms

Process returned 0 (0x0)   execution time : 0.025 s
Press any key to continue.

```

3. Write a C program to generate the area of an irregular figure using monte-carlo method.

```
#include<bits/stdc++.h>

using namespace std;

int main()
{
    int i,xmin,xmax,ymin,ymax,n;
    float area,mf,r1,r2,m,a,x,y;
    cout<<"Enter the value of xmin: ";
    cin>>xmin;
    cout<<"Enter the value of xmax: ";
    cin>>xmax;
    cout<<"Enter the value of ymin: ";
    cin>>ymin;
    cout<<"Enter the value of ymax: ";
    cin>>ymax;
    cout<<"\nEnter the number of random number you want to generate: ";
    cin>>n;
    if(xmin<=0)
        xmax=xmax-xmin;
    if(ymin<=0)
        ymax=ymax-ymin;
    if(ymax>xmin)
        mf=ymax+10;
    else
        mf=xmax+10;
    for(i=1;i<=n;i++){
        x=rand()/32768.0;
        y=rand()/32768.0;
        x*=mf;
        y*=mf;
```

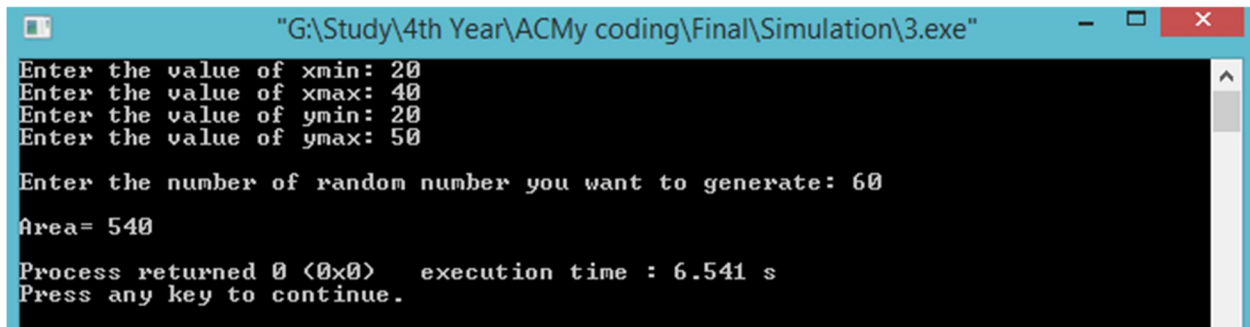


```

        if(x>=xmin&& x<=xmax&& y>=ymin&& y<=ymax)
            m++;
    }
    mf*=mf;
    area=m/float(n);
    area*=mf;
    cout<<"\nArea= "<<area<<endl;
    return 0;
}

```

Input & Output:



```

"G:\Study\4th Year\ACMy coding\Final\Simulation\3.exe"
Enter the value of xmin: 20
Enter the value of xmax: 40
Enter the value of ymin: 20
Enter the value of ymax: 50

Enter the number of random number you want to generate: 60

Area= 540

Process returned 0 (0x0)   execution time : 6.541 s
Press any key to continue.

```

4. Write a C program to determine the value of numerical integration using monte-carlo method.

```

#include<stdio.h>
#include<stdlib.h>
#include<math.h>
int n,i=1;
float r,x,y,x3,h,w,area,I,m,u;
int main()
{
    printf("Enter the number of observation: ");
    scanf("%d",&n);
    printf("Enter the height: ");

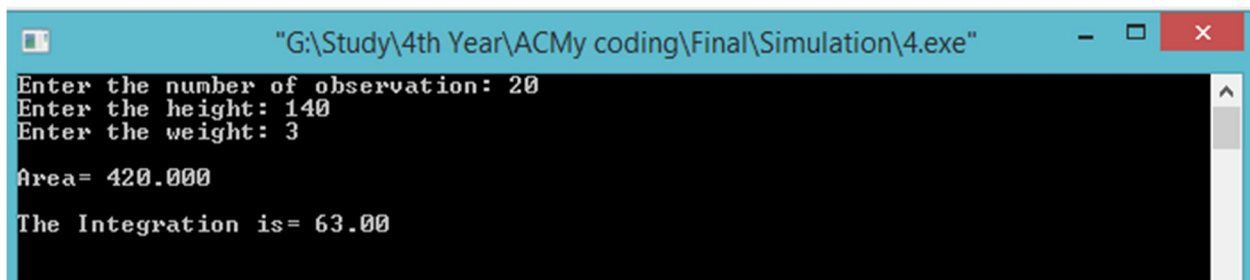
```

```

scanf("%f",&h);
printf("Enter the weight: ");
scanf("%f",&w);
area=h*w;
printf("\nArea= %.3f\n",area);
do{
    r=rand()/32768.0;
    r*=50;
    if(r>20&&r<50){
        x=0.1*r;
        y=rand()*140/32768.0;
        x3=x*x*x;
        if(y<=x3)
            m++;
        i++;
    }
}while(i!=n);
u=n;
I=(m/u)*area;
printf("\nThe Integration is= %.2f\n\n",I);
return 0;
}

```

Input & Output:



```

G:\Study\4th Year\ACMy coding\Final\Simulation\4.exe
Enter the number of observation: 20
Enter the height: 140
Enter the weight: 3

Area= 420.000
The Integration is= 63.00

```

N.B: For this program can have different result on your console because different compiler can generate different random numbers.

5. Write a C program to determine the value of π by monte-carlo method.

```
#include<bits/stdc++.h>

using namespace std;

int main()
{
    int n,i;

    float r1,r2,m=0,p,ra;

    cout<<"How many numbers you want to generate: ";

    cin>>n;

    for(i=1;i<=n;i++){

        r1=rand()/32768.0;

        r2=rand()/32768.0;

        ra=sqrt(1- pow(r1,2));

        if(r2<=ra)

            m++;

    }

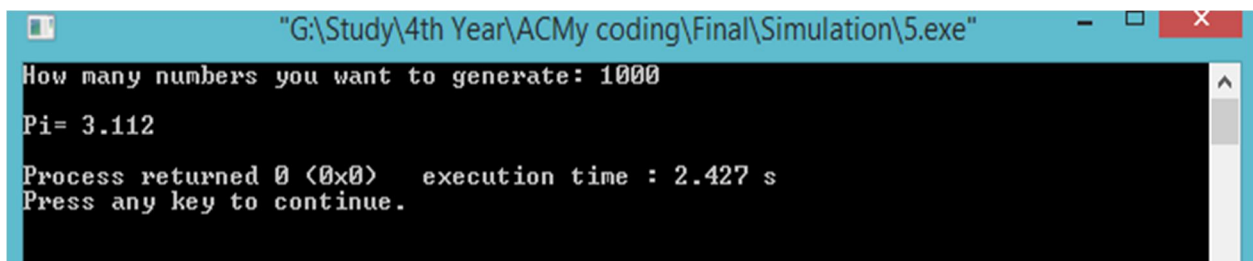
    p=(m/float(n)*4);

    cout<<"\nPi= "<<p<<endl;

    return 0;

}
```

Input & Output:



```
"G:\Study\4th Year\ACMy coding\Final\Simulation\5.exe"
How many numbers you want to generate: 1000
Pi= 3.112
Process returned 0 (0x0)   execution time : 2.427 s
Press any key to continue.
```

6. Write a C program for Mid-square Method of Generating 4-digit Random Numbers.

```
#include<stdio.h>

#include<stdlib.h>

#include<math.h>

int main()

{

    long seed=6785,a,b,c,i,j,k,x,y,z;

    for(i=1;i<=30;i++){

        y=seed*seed/100.0;

        z=y/10000.0;

        x=((y/10000.0)-z)*10000);

        seed=x;

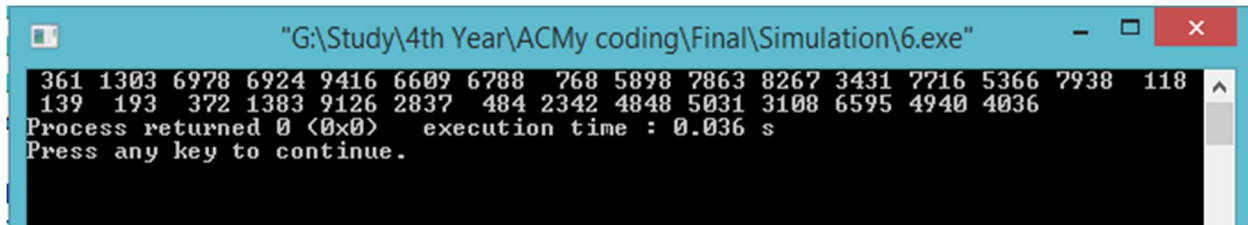
        printf("%4ld ",x);

    }

    return 0;

}
```

Input & Output:



```
"G:\Study\4th Year\ACMy coding\Final\Simulation\6.exe"
361 1303 6978 6924 9416 6609 6788 768 5898 7863 8267 3431 7716 5366 7938 118
139 193 372 1383 9126 2837 484 2342 4848 5031 3108 6595 4940 4036
Process returned 0 (0x0) execution time : 0.036 s
Press any key to continue.
```

7. Write a C program to generate variables from NEGATIVE BINOMIAL distribution having Parameters k (number of successes) and p (probability of success in trial).

```
#include<bits/stdc++.h>

using namespace std;

int main()

{

    int i,k,s,x,n;

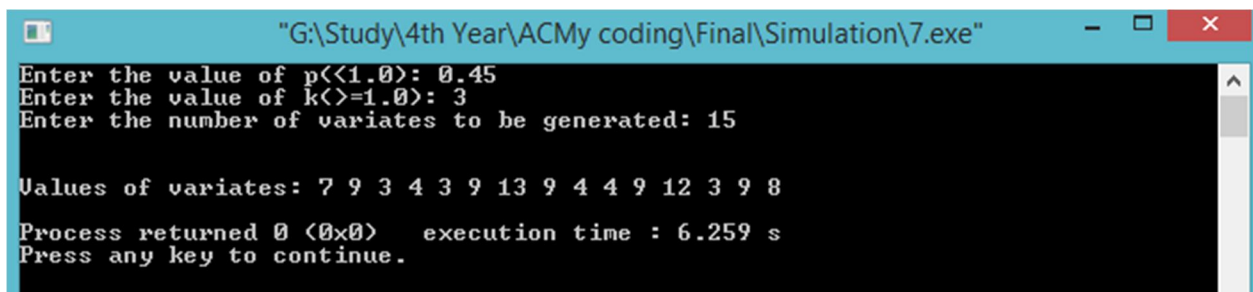
    float p,r;
```

```

printf("Enter the value of p(<1.0): ");
scanf("%f",&p);
printf("Enter the value of k(>=1.0): ");
scanf("%d",&k);
printf("Enter the number of variates to be generated: ");
scanf("%d",&n);
printf("\n\nValues of variates: ");
for(i=1;i<=n;i++){
    x=0;
    s=0;
    while(s<k){
        r=rand()/32768.0;
        if(r<=p)
            s++;
        x++;
    }
    printf("%d ",x);
}
printf("\n");
return 0;
}

```

Input & Output:



```

"G:\Study\4th Year\ACMy coding\Final\Simulation\7.exe"
Enter the value of p(<1.0): 0.45
Enter the value of k(>=1.0): 3
Enter the number of variates to be generated: 15

Values of variates: 7 9 3 4 3 9 13 9 4 4 9 12 3 9 8
Process returned 0 (0x0)   execution time : 6.259 s
Press any key to continue.

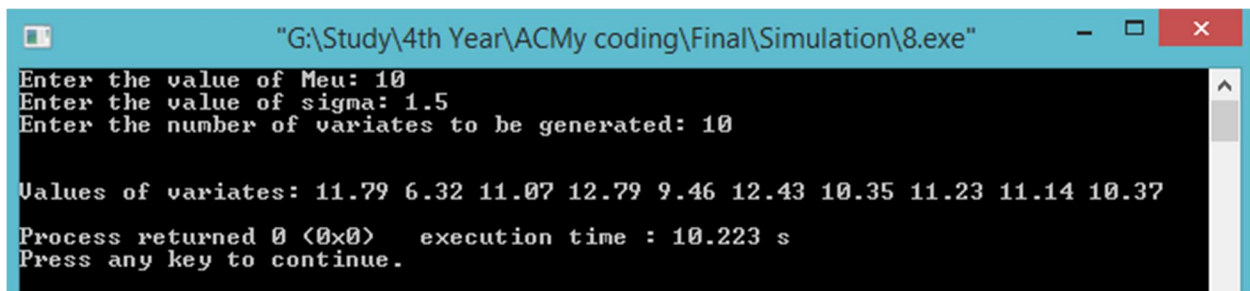
```

8. Write a C program to generate random variable from a Normal distribution having mean μ and standard deviation σ .

```
#include<stdio.h>
#include<stdlib.h>

int main()
{
    int i,j,s,n;
    float meu,sigma,sum,x,t;
    printf("Enter the value of Meu: ");
    scanf("%f",&meu);
    printf("Enter the value of sigma: ");
    scanf("%f",&sigma);
    printf("Enter the number of variates to be generated: ");
    scanf("%d",&n);
    printf("\n\nValues of variates: ");
    for(i=1;i<=n;i++){
        sum=0;
        for(j=0;j<=12;j++){
            x=rand()/32768.0;
            sum+=x;
        }
        t=meu+sigma*(sum-6);
        printf("%.2f ",t);
    }
    printf("\n");
    return 0;
}
```

Input & Output:



A screenshot of a Windows command prompt window. The title bar is light blue and contains the text "G:\Study\4th Year\ACMy coding\Final\Simulation\8.exe" along with standard window controls (minimize, maximize, close). The command prompt area has a black background with white text. It shows three input prompts: "Enter the value of Meu: 10", "Enter the value of sigma: 1.5", and "Enter the number of variates to be generated: 10". Below these, it displays the output: "Values of variates: 11.79 6.32 11.07 12.79 9.46 12.43 10.35 11.23 11.14 10.37". At the bottom, it shows "Process returned 0 (0x0) execution time : 10.223 s" and "Press any key to continue.".

```
"G:\Study\4th Year\ACMy coding\Final\Simulation\8.exe"  
Enter the value of Meu: 10  
Enter the value of sigma: 1.5  
Enter the number of variates to be generated: 10  
  
Values of variates: 11.79 6.32 11.07 12.79 9.46 12.43 10.35 11.23 11.14 10.37  
  
Process returned 0 (0x0) execution time : 10.223 s  
Press any key to continue.
```